

Занятие 7. Синтез: как разбирать новую архитектуру

Что делать, когда выходит новая paper с незнакомым названием и обещанием «faster / longer / cheaper»?

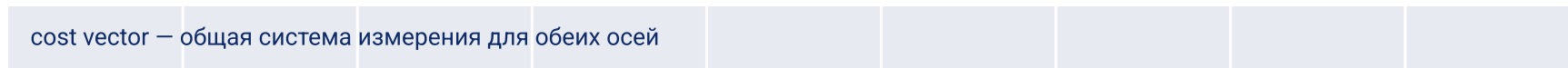
Автор: Андрей Ильинский, 2026

Весь курс — две оси и одна система стоимости

ось A — контекст и backbone



ось B — генеративная конструкция



в центре не названия моделей, а вопросы к механизму

Cost vector возвращается целиком

$$\mathcal{C} = (M_{\text{weights}}, M_{\text{persistent}}, M_{\text{workspace}}, B_{\text{HBM}}, F, D_{\text{seq}}, N_{\text{FE}}, N_{\text{processed}}, B_{\text{max}}).$$

M_w weights	M_p persistent	M_{ws} workspace	B_{HBM} traffic	F flops	D_{seq} sequential	N_{FE} calls	N_{proc} tokens	B_{max} batch
------------------	---------------------	-----------------------	-----------------------------	--------------	--------------------------------	--------------------------	-----------------------------	---------------------------

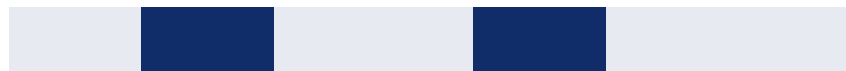
$$N_{\text{processed}} = \sum_{j=1}^{N_{\text{FE}}} L_j.$$

Одинаковая одна координата не задаёт стоимость

одинаковый 8 GiB KV pool



dense read всего pool



sparse main read: только subset

одинаковый $N_{FE} = 16$



full sequence: 1024 positions на вызов



current block: 64 positions на вызов

$$N_{FE} \not\approx N_{\text{processed}} \not\approx F \not\approx \text{latency}.$$

Три смысла слова «эквивалентность»

1. operator equivalence

две записи задают одну function

$$y_t = \sum_{i=1}^t (q_i^\top k_i) v_i.$$

2. computational equivalence

разные algorithms вычисляют один operator



parallel · recurrent · chunkwise

3. architectural similarity

общий design principle, не equality models

additive → forgetting → targeted editing →
finer-grained control

Декомпозиция

Какие семь вопросов задать любому механизму с состоянием?

Online-learning interpretation delta rule — точная, но локальная

$$\ell_t(S) = \frac{1}{2} \|Sk_t - v_t\|^2.$$

$$S_t = S_{t-1} - \beta_t (S_{t-1}k_t - v_t)k_t^\top.$$

$$S_t = S_{t-1} + \beta_t (v_t - S_{t-1}k_t)k_t^\top.$$

inner update — fast state

outer training objective network weights — отдельный уровень

Карточка новой architecture — mechanism и algorithm

1. Какую задачу решает layer или generative mechanism?

2. Каково основное equation?

3. Что хранится как state или cache?

4. Как выполняется read?

5. Как выполняется write/update?

6. Что забывается?

7. Есть ли parallel, recurrent или chunkwise representation?

8. Какова training complexity?

9. Какова decode/generation complexity?

10. Какие hardware assumptions используются?

Карточка — generative construction и falsification

11. Какая precision предполагается?

12. Какой objective используется?

13. Какой probability/noise path?

14. Какой sampler?

15. Сколько N_{FE} ?

16. Сколько элементов на один NFE?

17. С каким baseline сравнивают?

18. Что доказано exact identity или theorem?

19. Что только измерено авторами?

20. Какие failure modes или bottlenecks остаются?

21. Как попытаться опровергнуть понимание: shapes, orientation, causal dependencies, limiting cases, independent reference, outputs, gradients, precision, hardware profile?

Как читать статью

Что проверить раньше, чем поверить бенчмарку?

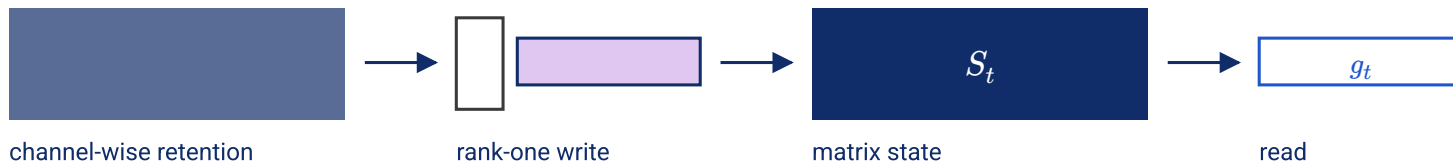
Доказано и измерено — разные колонки

ДОКАЗАНО / СЛЕДУЕТ ИЗ ОПРЕДЕЛЕНИЯ	ИЗМЕРЕНО
Causal sum равна выведенной recurrence	Throughput конкретного kernel
Fixed-shape state не растёт с T при фиксированных dimensions	Реальная peak GPU memory
Exact causal dependency разрешает reuse past KV	Tokens/s конкретной implementation
Chunkwise decomposition вычисляет тот же operator	Benchmark accuracy
Block-causal dependency делает completed prefix invariant	Ускорение block diffusion в implementation авторов
Конкретная asymptotic оценка при заданном algorithm	Latency на конкретном accelerator

Method X: сначала equation, потом название

$$S_t = D_t S_{t-1} + u_t v_t^\top, \quad y_t = g_t^\top S_t.$$

$$S_t \in \mathbb{R}^{d_s \times d_v}, \quad D_t = \text{diag}(d_t), \quad d_t \in (0, 1)^{d_s}, \quad u_t, g_t \in \mathbb{R}^{d_s}, \quad v_t \in \mathbb{R}^{d_v}.$$



Этого примера в курсе не было: проверяем навык, а не память.

Method X по карточке

State — fixed-shape matrix S_t

Read — $g_t^\top S_t$

Write — rank-one $u_t v_t^\top$

Forgetting — channel-wise diagonal D_t

Training algorithm — parallel scan no diagonal transitions

Decode algorithm — recurrent

Ось A — finite context state + channel-wise forgetting

Ось B — autoregressive factorization; не следует из recurrence layer

Cost claim — fixed persistent state scaling; latency не доказана одной formula

Не только понять — попытаться сломать своё объяснение

$$D_t = I$$

forgetting должно исчезнуть

shapes и orientation

causal dependency

limiting cases

independent naive reference

compare outputs

compare gradients

precision sensitivity

hardware profile

Две optimized implementations, совпавшие между собой, не являются независимым proof correctness.

Сводка курса — explicit cache и finite state

КЛАСС	STATE / CACHE	READ / UPDATE	ГЛАВНАЯ COST AXIS
Full causal attention	token-indexed KV-cache	softmax read past K/V; append K/V	$M_{\text{persistent}}, B_{\text{HBM}}, F$
GQA / MQA	KV-cache с меньшим числом distinct KV-heads	query-heads share K/V; append shared K/V	$M_{\text{persistent}}, B_{\text{HBM}}$
Compressed KV / MLA-like	token-indexed compressed latent past	architecture-specific reconstruction/read	$M_{\text{persistent}}, B_{\text{HBM}}, F$
Additive linear state	$S_t \in \mathbb{R}^{d_v \times d_k}$	$S_t q_t; S_t = S_{t-1} + v_t k_t^\top$	$M_{\text{persistent}}, D_{\text{seq}}$
Decayed state / retention	fixed-shape state	read decayed state; retention + write	$M_{\text{persistent}}, D_{\text{seq}}, F$
Delta rule	fixed matrix state	read state; write prediction error	state capacity, F, D_{seq}

Сводка курса — управляемое state, hybrids и sparse read

КЛАСС	STATE / CACHE	READ / UPDATE	ГЛАВНАЯ COST AXIS
Gated DeltaNet	fixed matrix state	global α_t + targeted β_t correction	$M_{\text{persistent}}, F, D_{\text{seq}}$
SSD / Mamba-2	structured finite state	structured state transition / output map	$F, D_{\text{seq}}, B_{\text{HBM}}$
RWKV-7	finite recurrent state	generalized delta-like update	state utilization, F, D_{seq}
KDA	finite recurrent state	delta-family editing + finer-grained gating	state utilization, F, B_{HBM}
Hybrid state + attention	fixed states + token-indexed KV-cache	разные layers/branches читают разную memory	весь profile, особенно $M_{\text{persistent}}$
Sparse attention	большой token-indexed KV pool	selected blocks / positions	B_{HBM}, F , иногда $M_{\text{persistent}}$

Сводка курса — generative constructions

КЛАСС	BACKBONE / PERSISTENT CONTEXT	GENERATIVE STATE / UPDATE	ГЛАВНАЯ COST AXIS
Flow matching	зависит от backbone	current sample x_t ; ODE solver	N_{FE}, F, D_{seq}
Gaussian diffusion / score	зависит от denoising backbone	noisy sample; reverse sampler	N_{FE} , per-NFE F, D_{seq}
Masked diffusion	exact static full-sequence KV-cache обычно нет	current corrupted sequence; iterative unmasking	$N_{FE}, N_{processed}, F$
Block diffusion	KV-cache completed immutable blocks	diffusion current block; block becomes immutable prefix	$N_{FE}, N_{processed}, M_{persistent}, B_{HBM}$

Строки «лучшая architecture» нет

state/cache → read → write → forgetting → algorithm → cost vector → objective/path/sampler → proof vs measurement.

**Курс не отвечает, какая архитектура лучшая.
Он даёт вопросы, которые задают незнакомой
архитектуре.**

- 1 — внимание → конечное состояние
- 2 — состояние → редактируемое состояние
- 3 — редактируемое состояние → гибридный и разреженный доступ
- 4 — дискретная генерация → непрерывный перенос
- 5 — перенос → score и диффузия
- 6 — диффузия → дискретная генерация и возврат кэша

Автор: Андрей Ильинский, 2026